24. Acceptance and QA Checklist

Cross-layer acceptance gates for shipping work on QuantPipeline

1. Why this checklist exists

QuantPipeline spans product logic, recommendation policy, data models, API contracts, UX, and admin operations. A phase can appear successful in one layer while being broken in another. This checklist forces a gate-based decision instead of a vibes-based one.

Acceptance Gate Stack

	Question	Failure mode if skipped
Requirements gate	Did output match requested scope?	Spec drift and hidden omissions
Architecture gate	Is behavior consistent across layers?	Pretty docs masking broken architecture
Data/API gate	Do entities and contracts line up?	Schema/API contradictions
UX gate	Are key user paths covered?	Feature exists but unusable in real work
Evidence gate	Is proof stronger than claims?	False confidence and trust erosion
Packaging gate	Can another model continue safely?	Handoff unusable despite strong content

2. Master acceptance gates

Gate	Acceptance question	Minimum evidence
Requirements	Did the delivered scope match the requested scope?	Scope table plus touched-file list
Architecture	Do the implemented surfaces	Architecture trace and code

	match the intended responsibilities?	inspection
Data model	Are entities, fields, enums, and derived outputs coherent?	Schema review and sample records
API contract	Do request/response payloads match the contract docs and error semantics?	Endpoint proofs and contract mapping
UX/UI	Are the primary user journeys usable, comprehensible, and aligned with docs 16–20?	Screenshots or live inspection of main states
Admin/ops	Can an operator understand, control, and audit the relevant workflow?	Operator proof and audit evidence
Evidence packaging	Can another engineer/model continue safely from the package?	Ordered package with README and known gaps

3. Implementation checklist

- Phase goal stated in one sentence.
- In-scope and out-of-scope clearly listed.
- Exact files/services/routes/tables/components named.
- No hidden broad refactor without explanation.
- Changed code is internally consistent and free of placeholder logic.

4. Data and contract checklist

- Canonical entity names match document 11.
- Field names and types are coherent across storage, service, and API layers.
- Enums carry consistent semantics in code and docs.
- Recommendation-object structure remains stable and explainable.
- Validation and error responses are explicit rather than implicit.

5. UX and journey checklist

- Primary journey can be completed without hidden operator knowledge.
- Trust cues and confidence states are present where decisions are made.
- Desktop and mobile information hierarchy remain usable.
- Empty, loading, error, and degraded-data states are designed intentionally.
- Action labels are concrete and decision-oriented.

6. Review and evidence checklist

- All major claims map to inspected evidence.
- Tests or probes used are appropriate for the change type.
- Rendered docs or screenshots were visually checked, not only generated.
- Unverified items are explicitly labeled.
- PASS / PARTIAL / FAIL is assigned against named gates, not overall sentiment.

7. Release-readiness checklist

- README or handoff file explains order and context.
- Package contains exact current artifacts, not stale placeholders.
- Known limitations are written plainly.
- The next phase is bounded and can start without rediscovering basic truth.
- If the package is partial, that is stated in the first page or summary.

8. Example gate decision table

Gate	Status	Why
Requirements	PASS	Delivered exactly the bounded scope
Data/API	PARTIAL	One optional field not yet wired in responses
UX/UI	PASS	Primary and error states shown in evidence
Evidence packaging	FAIL	No README and no known-gaps section

9. Final ship rule

A phase should ship only when blockers are closed or explicitly accepted by the operator. Cosmetic issues can remain open, but contradictions across requirements, architecture, data, API, or primary journeys cannot be hidden under a generic 'done' statement.

10. Checklist usage modes

Mode	Use when	Output
Phase-end QA	After a bounded build pass	Gate table plus defect list
Release gate	Before handing to another model/engineer	Go / no-go package decision
Recovery mode	After partial crash or interrupted run	What exists, what is missing, next smallest safe step